NIET Greater Noida

An Autonomous Institute

Program and Branch Name: B.Tech CSE (Cyber Security)

PBL Final Report

Field	Details
Course Name	Cryptography
Course Code	CCSCY0301
Faculty Name	Dr. Rashmi Sharma
Assignment Type	Project-Based Learning (PBL) Report - Final
Student Name	Ayush Jha
ERP ID / Roll No.	0251CYS120 / 2501331720040
Project Title	Secure Product Tracking System
SDG(s)	SDG 9 (Industry, Innovation and Infrastructure) and SDG 12 (Responsible Consumption and Production)
Source code	paste your GitHub repository link here
Date of Submission	date of submission

Student Declaration

I certify that this assignment is my own work, with all sources appropriately acknowledged. I understand that a 0% mark and further academic penalties may apply if the content is found to be plagiarized or inappropriately AI-generated.

Student Name & Signature: Ayush Jha sign here

Contents

1. Abstract

2. Introduction

3. Objectives and Scope

4. Related Work (what already exists)

5. Cryptography Background Used

6. System Design

7. Implementation

8. Experiments and Results

9. Testing

10. Limitations

11. Link to SDG 9 and SDG 12

12. Conclusion and What I Learned

References

Appendix A: Demo output

Appendix B: How to run the project

Appendix C: Repository structure

1. Abstract

When a product moves from a factory to a shop, many different people write down what happened to it. If those records are kept in spreadsheets or ordinary databases, anyone with access can change them quietly, and a customer has no way to check whether the history is real. In this project I built a small tracking system in Python that tries to fix this one problem: making the tracking history hard to change without being noticed.

Each tracking step (registered, shipped, received, sold) is saved as an event. The event stores the hash of the previous event, so the events form a chain, and it is signed with the private key of the person who created it. Access is controlled by roles, and the order of steps is checked so that, for example, a product cannot be sold before it was received. A customer can enter the product ID and label code to verify a product, and an admin can audit the whole chain. Everything can be used through a small website (built with Flask) or from the command line.

I also tested the system. In 600 simulated attacks on a 300-event chain (editing a record, deleting a record, and editing a record while recomputing its hash) every attack was detected. I also showed experimentally why the classical ciphers from Module 1 (Caesar and substitution) are not suitable for this job: a simple frequency analysis recovered the Caesar key in every trial once the text was 40 letters or longer. The system has clear limits, listed in Section 10; the most important one is that private keys are stored on the same machine as the database.

2. Introduction

2.1 Problem statement

A product usually passes through several hands: manufacturer, distributor, retailer and finally the customer. Every handover creates a record. The problems I wanted to look at are:

1. A record can be changed after it was written (for example, a shipment location or date).
2. A fake product can carry a copied or invented product number.
3. Different parties keep separate records, so the full journey is difficult to see in one place.
4. There is no trustworthy audit trail showing who did what and when.

This is the same set of problems I listed in my first progress report (unauthorised modification, counterfeit products, fragmented records, slow tracing and no reliable audit trail). In this final report I describe the prototype I built to address them and the tests I ran.

2.2 Why cryptography?

The course Cryptography (CCSCY0301) covers confidentiality, integrity and authentication. For product tracking, integrity and authentication matter more than confidentiality: the main question is not "can someone read the record" but "can someone change it without us

noticing, and who wrote it?". Hash functions and digital signatures answer exactly those two questions, so they became the core of the project.

3. Objectives and Scope

3.1 Objectives

- Store product movement as a history that cannot be edited or deleted without detection.
- Show who created each record by using digital signatures.
- Allow only the right role to do the right step (least privilege).
- Let a customer check a product using its ID and label code.
- Test the system and measure how well tampering is detected.
- Understand, with experiments, why classical ciphers are not enough.

3.2 What is in scope and what is not

Planned (from progress reports)	Status in final prototype
SRS, ER schema, UI wireframes (Report I)	Done earlier. Schema reflected in ledger.py (users, products, events).
SHA-256 hashing / integrity service	Done - ledger.py
Role-based access control	Done - auth.py, service.py
JWT-style authentication	Done as a hand-written HMAC token with expiry (same idea as JWT, simplified)
Product verification using unique ID / QR	Done as ID + HMAC label code. QR image generation and scanning NOT done.
Secure REST APIs	Not built as a separate REST API. A Flask website (app.py) and a command line interface use the same service layer instead.
UI wireframes (Report I)	Built as a working website: home, verify, login, dashboard, audit, tamper demo
Digital signatures (conceptual in Report II)	Implemented with Ed25519
Classical ciphers (Report II)	Implemented and attacked in Experiment 1

I am listing the missing parts here on purpose. They are possible future work, and I did not want the report to claim more than the code does.

4. Related Work (what already exists)

4.1 GS1 EPCIS - the standard way to describe tracking events

EPCIS (Electronic Product Code Information Services) is a GS1 standard for capturing and sharing event data about products as they move through a supply chain [1]. Each event records what happened, where, when and why. I borrowed this idea for my event structure: product_id (what), location (where), timestamp (when), action (why) and actor (who). EPCIS

is mainly about a shared data format; it does not by itself make records tamper-evident, which is the part I focused on.

4.2 Open-source projects using hash chains

To see how others do tamper-evident records, I read several public GitHub projects. I did not copy their code; I used them to understand the design idea and then wrote my own implementation.

Project	What I noticed	What I did differently
ForgeProof [2]	Signs each receipt with Ed25519 and links entries in a SHA-256 hash-chained ledger.	Same primitives, but applied to product movement events with role rules and a lifecycle state machine.
nexuscone [3]	Append-only audit ledger with SHA-256 chain and optional Ed25519 signing.	I made signing mandatory and tied each signature to the user's role.
AI-Risk-Based-Inventory-ERP [4]	Uses a hash-chain audit log inside an ERP workflow and states that the audit is tamper-evident, not tamper-proof against a database administrator.	I follow the same honest wording and add an external anchor hash for tail-deletion.
pipewatch [5]	Uses HMAC-signed baselines to make stored reference data tamper-evident.	I used HMAC for the product label code and for session tokens.

4.3 Standards and references used for the cryptography

SHA-256 follows FIPS 180-4 [6]. Digital signatures use Ed25519 as specified in RFC 8032 [7]. HMAC follows RFC 2104 [8]. The token format is inspired by JWT (RFC 7519) [9]. Password storage follows the idea in NIST SP 800-132 (PBKDF2 with a salt and many iterations) [10]. For common web risks I used the OWASP Top 10 as a checklist [11].

5. Cryptography Background Used

5.1 Security goals

Goal	Meaning in this project	Tool used
Confidentiality	Keeping data secret. Not a main goal here.	(not used for events)
Integrity	Detecting changes to tracking records.	SHA-256 hash chain
Authentication	Knowing who created a record / who is logged in.	Ed25519 signatures, PBKDF2 passwords, HMAC tokens
Authorisation	Only allowed roles can do an action.	Role table (RBAC)

Traceability	Full ordered history of a product.	Event ledger

5.2 Classical ciphers (Module 1)

A Caesar cipher shifts every letter by a fixed number k. There are only 25 useful keys, so it can be broken by trying all of them. A monoalphabetic substitution cipher replaces each letter using a fixed random table. It has 26! possible keys, which is too many to try one by one, but it still keeps the letter frequencies of the language. In English, E, T and A are the most common letters, so an attacker can match the most common cipher letters with the most common English letters. I implemented both ciphers and a frequency attack (crypto_basics.py) and measured how well the attack works (Section 8.1).

5.3 Hash functions

A hash function such as SHA-256 turns any input into a fixed 256-bit output. Two properties matter for this project: (1) it is practically impossible to find another input with the same hash, and (2) a tiny change in input completely changes the output (the avalanche effect). Because of (2), if any field of an event is edited, its hash changes.

5.4 Digital signatures and HMAC

A digital signature uses a private key to sign data and a public key to check it. If an event is signed by the manufacturer's private key, anyone with the manufacturer's public key can check that the event really came from that account and was not modified. HMAC is similar but uses one shared secret key; it is cheaper and is good for things the server checks itself, like the label code and session tokens.

6. System Design

6.1 Architecture

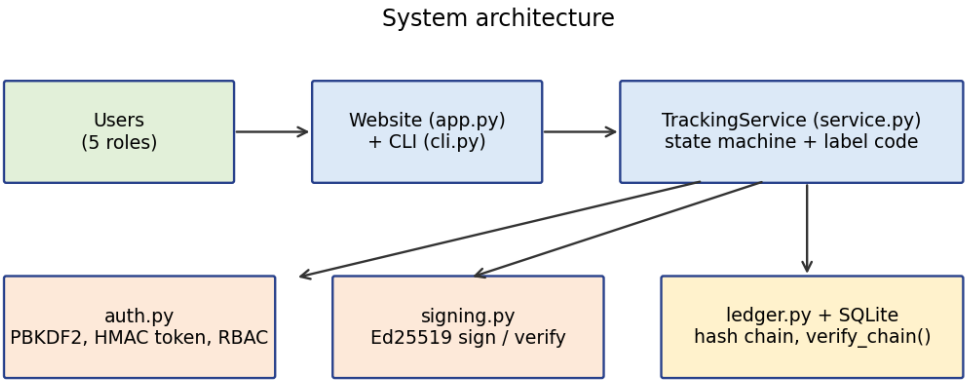


Figure 1: Architecture of the prototype

The user talks to the system through the website (app.py) or the command line (cli.py). The service layer (service.py) checks the role and the lifecycle rules, then asks the ledger (ledger.py) to store the event. The ledger asks signing.py to sign it. auth.py handles passwords, tokens and the role table.

6.2 Roles and permissions

Role	**register**	**ship**	**receive**	**sell**	**verify / history**	**audit**
manufacturer	yes	yes	-	-	yes	-
distributor	-	yes	yes	-	yes	-
retailer	-	-	yes	yes	yes	-
admin	-	-	-	-	yes	yes
customer	-	-	-	-	yes	-

6.3 Product lifecycle

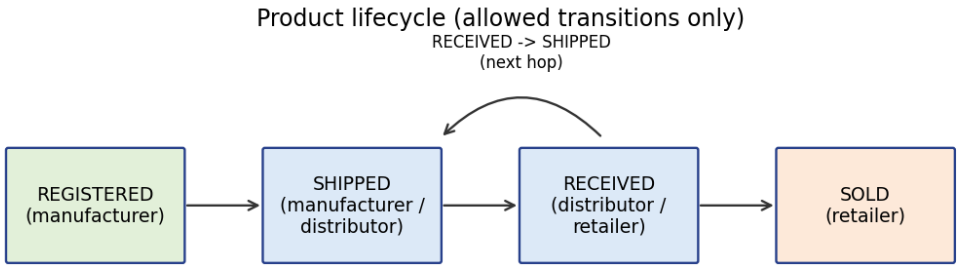


Figure 2: Allowed steps in a product's life

Only the transitions in Figure 2 are accepted. For example, a product that is only REGISTERED cannot be marked SOLD. This stops a dishonest user from inserting a "sold" record for a product that never reached the shop.

6.4 Hash-chained, signed events

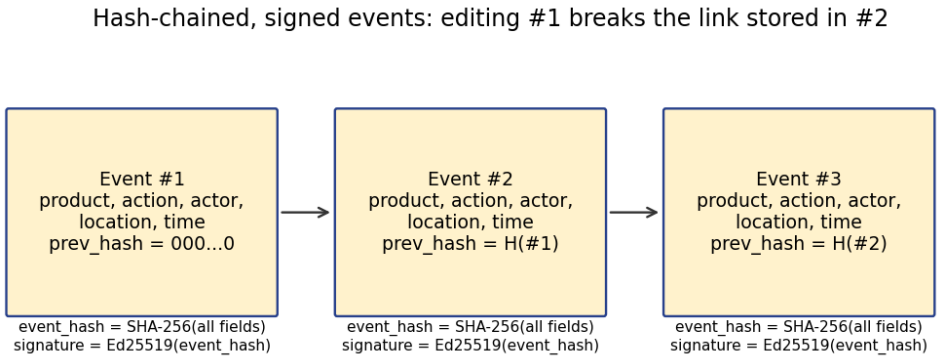


Figure 3: Each event contains the hash of the previous event

The hashed text is the event fields written as JSON with sorted keys, so the same event always produces the same bytes. The event hash is then signed. The first event uses a hash of 64 zeros as its previous hash.

```
event_hash = SHA-256( product_id, action, actor, location, note,
                      timestamp, prev_hash )          # canonical JSON
signature  = Ed25519_sign( actor_private_key, event_hash )
```

6.5 Database tables

Table	Columns
users	username, role, pw_salt, pw_hash, public_key
products	product_id, name, batch, created_by
events	seq, product_id, action, actor, location, note,

	timestamp, prev_hash, event_hash, signature

6.6 Verification by a customer

The product label carries the product ID and a label code. The label code is the first 10 characters of HMAC-SHA256(server_secret, product_id). Someone who invents a product ID cannot compute a matching code without the secret. After the code is checked, the system also verifies the whole chain; if the history was tampered with, the product is reported as not authentic.

6.7 Threat model

Threat	Defence	Remaining risk
Edit an old record	Event hash + chain link	-
Edit and recompute hashes	Ed25519 signature no longer matches	Attacker who steals the actor's private key
Delete a record in the middle	Next event's prev_hash no longer matches	-
Delete the newest records	External anchor hash saved outside the DB	Anchor must be kept safe
Fake product label	HMAC label code	Copying a real label onto a fake product
User does something outside their job	Role table + state machine	Stolen account
Stolen user table	PBKDF2-SHA256, salt, 200,000 rounds	Weak passwords
Forged or old token	HMAC signature + expiry (15 min)	Token theft inside 15 minutes

7. Implementation

The project is written in Python 3. The external libraries are cryptography (for Ed25519) and Flask (for the website [15]); everything else uses the standard library (hashlib, hmac, sqlite3, argparse). matplotlib is used only for the experiment graphs.

File	What it does
sptrack/crypto_basics.py	Caesar, substitution cipher, frequency attack, SHA-256 helper
sptrack/signing.py	Ed25519 key generation, sign, verify
sptrack/auth.py	PBKDF2 passwords, HMAC token, role permissions
sptrack/ledger.py	Event storage, hash chain, verify_chain()
sptrack/service.py	Lifecycle rules, label code, product verification
sptrack/cli.py	Command line commands
sptrack/demo_data.py	Demo accounts and sample products

app.py, templates/, static/	The Flask website: pages, forms, login session, CSRF token, security headers
tests/	33 automated tests
experiments/	Scripts that produced the results in Section 8

7.1 The core idea in code

Appending an event takes the hash of the last stored event, builds the new event, hashes it and signs it:

```
def append(self, product_id, action, actor, location, note, private_pem):
    prev = self.last_hash()
    body = event_body(product_id, action, actor, location, note, timestamp,
prev)
    h    = sha256_hex(body)
    sig  = signing.sign(private_pem, h)
    # INSERT the event with prev_hash=prev, event_hash=h, signature=sig
```

Verifying walks through all events from the start and checks three things for each one: the stored prev_hash equals the previous event's hash, the recomputed hash equals the stored hash, and the signature is valid for the actor's public key.

```
for e in all_events():
    if e.prev_hash != prev:                    -> broken link
    if sha256(event_body(e)) != e.event_hash:  -> content edited
    if not verify(pubkey(e.actor), e.signature, e.event_hash): -> bad signature
    prev = e.event_hash
```

7.2 Design decisions I had to make

- One global chain instead of one chain per product, so that cutting out a whole product history is also detected (cost: verification checks every event).
- Ed25519 instead of RSA because keys and signatures are small and the library API is simple.
- A hand-written token instead of a JWT library, because I wanted to understand how the signature part works. A real project should use a reviewed library.
- An external "anchor" hash, because a chain by itself cannot show that the newest events were deleted.

Add 2-3 sentences in your own words about a design decision you personally struggled with or changed your mind about while building this.

7.3 The website

To make the project easier to show and to use, I built a small website on top of the same service layer. Users log in and see only the forms their role allows. Anyone can open the

verify page. The pages use ordinary HTML forms, with a few safety measures: every form carries a CSRF token, the login is locked for a minute after 5 wrong attempts, the session cookie is HttpOnly, and the pages are sent with security headers.

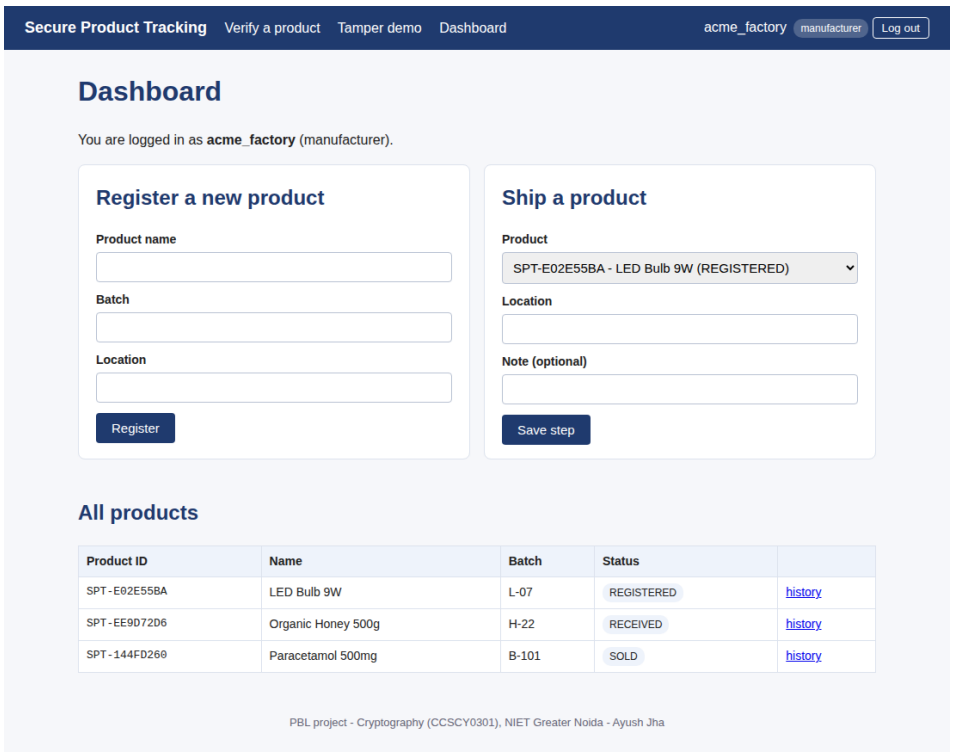


Figure 4: Manufacturer dashboard (only the allowed actions are shown)

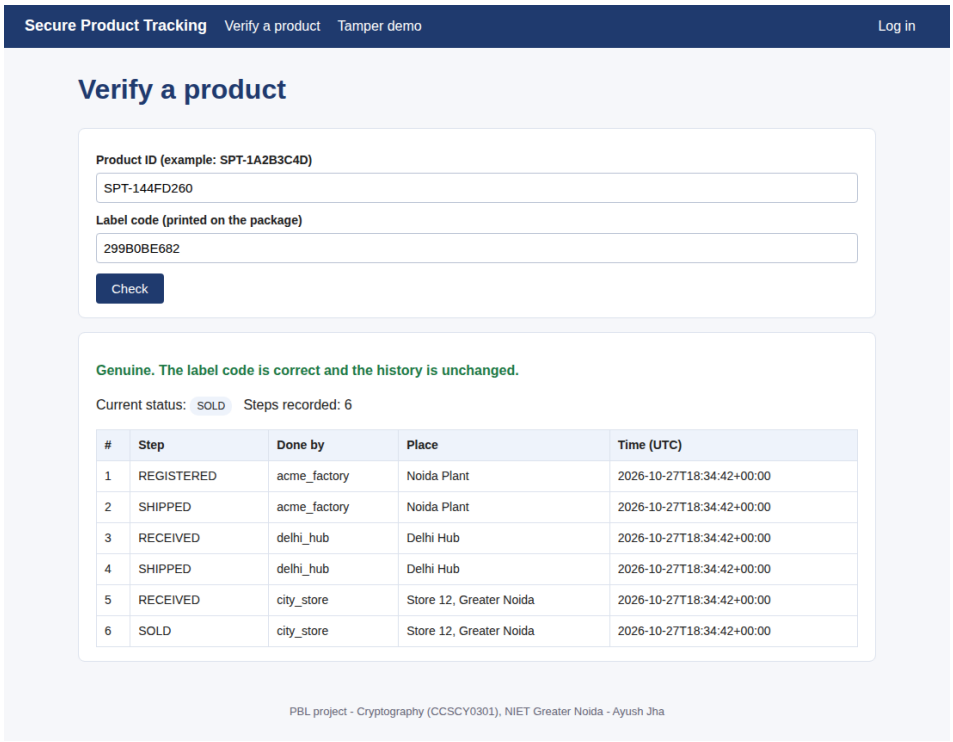


Figure 5: A customer verifies a genuine product

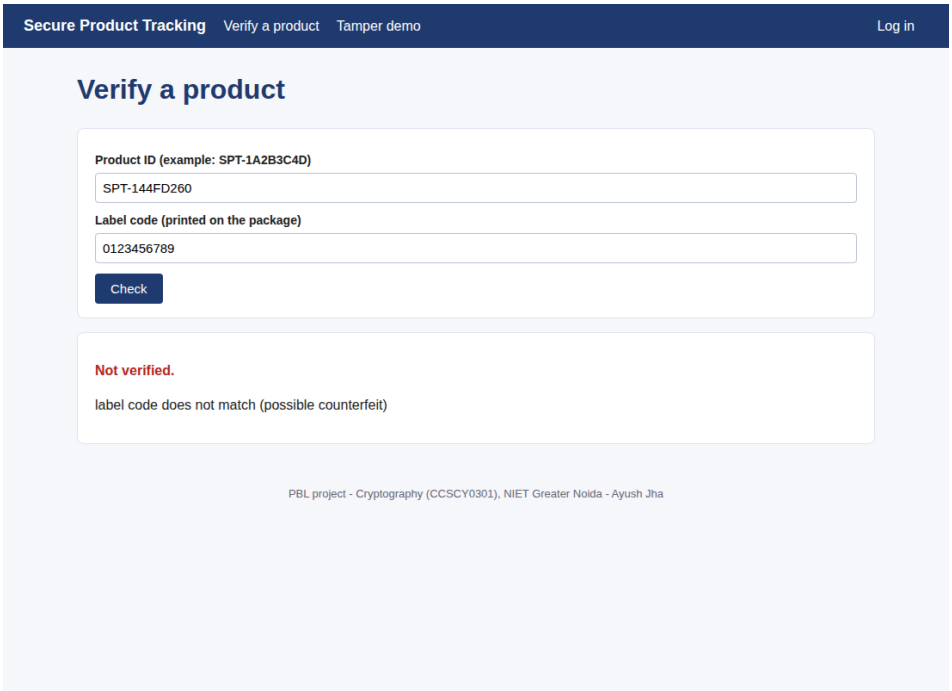


Figure 6: A wrong label code is rejected

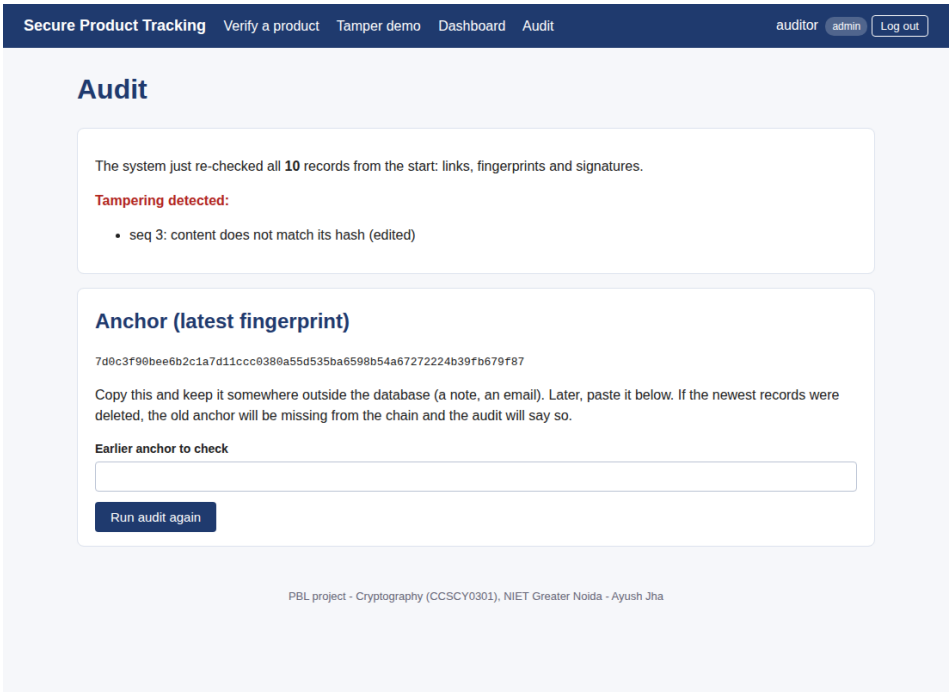


Figure 7: The admin audit after a record was edited directly in the database

The Tamper demo page runs an attack on a temporary copy of the database and shows the check failing, so the idea can be demonstrated without damaging the real data. The site can be deployed on a free host; the steps are in DEPLOY.md in the repository.

8. Experiments and Results

All experiments are in experiments/run_experiments.py and can be re-run. Numbers below are from one run on my machine; the timing numbers will be different on other computers.

8.1 Experiment 1: breaking classical ciphers

Setup: for each text length I took 200 random pieces of English text about product tracking, encrypted them with a random Caesar shift or a random substitution key, and attacked them. The Caesar attack tries all 26 shifts and picks the result whose letter frequencies are closest to English (chi-squared score). The substitution attack simply maps the most frequent cipher letters to the most frequent English letters.

Ciphertext length (letters)	Caesar key recovered	Substitution letters recovered
20	91.5%	15.9%
40	100.0%	20.1%
80	100.0%	28.1%
160	100.0%	32.2%
320	100.0%	33.1%

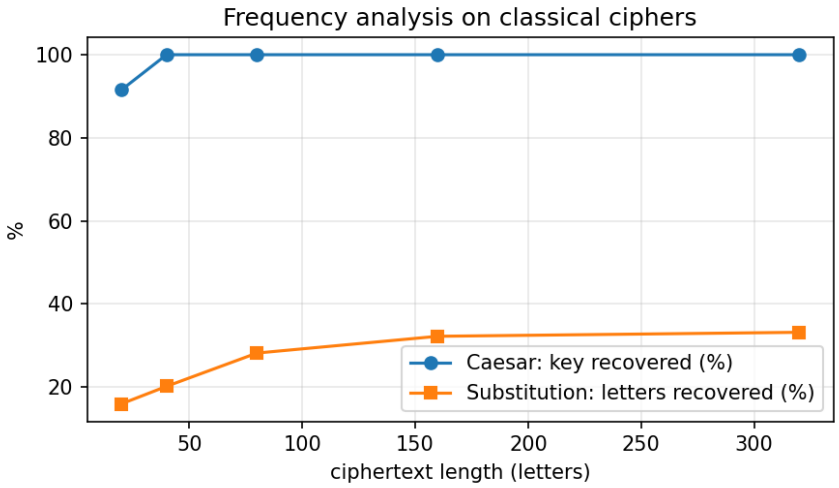


Figure 8: Frequency analysis on Caesar and substitution ciphers

Observations: the Caesar cipher was broken in every trial from 40 letters upward, and in 91.5% of trials even at 20 letters. The substitution cipher was harder: my very simple attack recovered only 16% to 33% of the letters, so it is a weak attack, not a full break. A better attack (using letter pairs and a search algorithm) would do much better, but I did not implement that. The main point for this project is different: even a cipher that resisted this attack would only hide the data. It would not tell us if someone changed the ciphertext. That is why the real system uses hashes and signatures.

8.2 Experiment 2: SHA-256 avalanche effect

I hashed 2000 pairs of event strings that differ by exactly one character and counted how many of the 256 output bits were different. The mean was 128.0 bits (ideal is 128), with a minimum of 100 and a maximum of 164. So a one-character edit changes about half of the hash bits, which is what makes tampering visible.

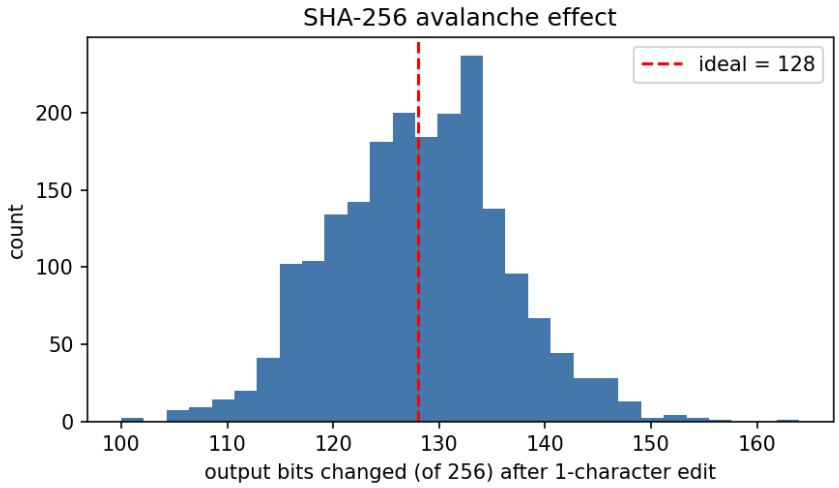


Figure 9: Number of output bits that change after a one-character edit

8.3 Experiment 3: tamper detection

I built a ledger with 300 events and, for each attack type, picked a random event 200 times, damaged it directly in the database, ran verify_chain() and then undid the change.

Attack on the database	Trials	Detected
Change the location field of a random event	200	200 (100%)
Delete a random event	200	200 (100%)
Change a field AND recompute that event's hash	200	200 (100%)

The third attack is still detected for two reasons: the next event still stores the old hash as its previous hash, and the signature on the edited event was made over the old hash. Here the attacker did not have the private key. If an attacker also steals the key, they could re-sign events, which is a limit I explain in Section 10. Another case, deleting the newest events, is not visible from the chain alone; I tested it separately in the unit tests and it is only caught when an anchor hash saved earlier is supplied.

8.4 Experiment 4: performance

Events in ledger (approx.)	Time to append one event (ms)	Time to verify the full chain (ms)
100	1.6	15
500	2.3	73
1000	1.9	136
2000	2.6	237

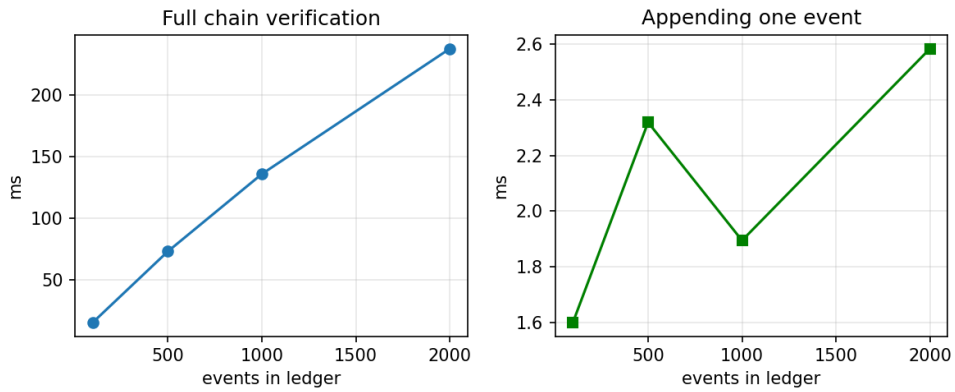


Figure 10: Append time and full verification time

Appending is quick and stays roughly flat. Full verification grows with the number of events because every signature is checked, and it took about a quarter of a second for roughly 2000 events. For a larger system the chain would have to be verified in parts (for example, checkpoints) instead of from the beginning every time. I did not test beyond a few thousand events.

9. Testing

I wrote 33 automated tests with Python's unittest module; all pass. They cover:

Area	Tests
Classical ciphers and hashing	Caesar round trip, known shift value, key recovery on long text; substitution round trip and key validity; the SHA-256 test vector for the string "abc"; avalanche range
Lifecycle	Full journey verifies as authentic; wrong label code rejected; unknown product rejected; skipping a step rejected
Access control	Retailer cannot register; customer cannot ship; wrong password rejected; forged token rejected; expired token rejected
Tamper detection	Clean chain passes; edited field detected; deleted middle event detected; edit-and-rehash detected; deleted newest event only detected with the anchor
Several users at once	Six threads adding events at the same time; the chain still verifies
Website	Pages open; login needed for the dashboard; wrong password; form without CSRF token rejected; each role limited to its own actions; customer cannot open audit; audit detects an edit; tamper demo leaves real data unchanged; security headers present; login lock after 5 failures

```
$ python -m unittest discover -s tests
...
Ran 33 tests
```

```
OK
```

9.1 Bugs the tests found

Two bugs showed up while testing the website, and I fixed both:

5. The CSRF check passed when a form was sent with no token at all, because "nothing" equalled "nothing" (the session also had no token yet). The check now requires a real token and compares it safely.
6. When I ran the real server, pages that used the database crashed, because the web server handles each request in a different thread and my SQLite connection could only be used by the thread that created it. I changed the connection setting and added a lock plus a database transaction around writing an event, so two users cannot add an event at the same instant and split the chain.

10. Limitations

7. Private keys are files on the same machine as the database. Someone who controls the server can sign forged events. A real deployment would keep keys on each company's own device or in a hardware module.
8. Deleting the newest events cannot be seen without an external anchor hash. The anchor must be stored somewhere an attacker cannot change (printed, emailed, or in a public repository).
9. The system is tamper-evident, not tamper-proof: it can detect changes but cannot prevent someone with database access from making them.
10. A real label can be photographed and copied onto a fake product. The label code proves that the ID is genuine, not that this physical item is the original. Solving that needs physical measures (tamper-evident seals, per-item secure tags).
11. Event contents are not encrypted. Anyone who can read the database can read the history.
12. No separate REST API, no QR code generation or scanning, no mobile app. The website and command line are the only interfaces.
13. Timestamps come from the server clock and are not independently verified.
14. Built for one small server: SQLite, one server process (the login lock-out counter is kept in memory), and the verification time grows with the chain size.
15. The token and password code is a learning implementation. It has not been security-reviewed.

11. Link to SDG 9 and SDG 12

SDG 9 is about building reliable infrastructure and encouraging innovation. A tracking system that organisations can trust is a small piece of that kind of digital infrastructure. SDG 12 is

about responsible consumption and production; being able to check where a product came from and whether it is genuine supports that goal, because it makes counterfeit and untraceable goods harder to pass off. This is a prototype, so I only claim that it demonstrates the technique, not that it achieves these goals by itself.

12. Conclusion and What I Learned

The project shows that a few standard cryptographic tools, SHA-256 chaining, Ed25519 signatures, HMAC and a role table, are enough to make a product's tracking history tamper-evident in a small system. All 600 simulated attacks on the stored history were detected, and the main remaining risks are about key storage and about physical labels, not about the hash chain itself.

The experiments also changed how I looked at Module 1. Breaking the Caesar cipher took only a few lines of code, and the substitution cipher result showed me that hiding data and protecting it from change are two different problems.

Add a short personal paragraph: what was hardest for you, what you would do next (e.g. add a REST API with Flask, QR codes, or key storage on a separate device), and one thing you understand now that you did not before.

References

[1] GS1, "EPCIS and CBV," GS1 standards. [Online]. Available: https://www.gs1.org/epcis

[2] bxrist, "ForgeProof," GitHub repository. [Online]. Available: https://github.com/bxrist/ForgeProof

[3] Aperintel, "nexuscone: tamper-evident append-only audit ledger," GitHub organisation page. [Online]. Available: https://github.com/Aperintel

[4] falltwo, "AI-Risk-Based-Inventory-ERP," GitHub repository and releases (v0.1, v1.0). [Online]. Available: https://github.com/falltwo/AI-Risk-Based-Inventory-ERP

[5] "pipewatch-cli," Python Package Index. [Online]. Available: https://pypi.org/project/pipewatch-cli/

[6] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, Aug. 2015.

[7] S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," IETF RFC 8032, Jan. 2017.

[8] H. Krawczyk, M. Bellare and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," IETF RFC 2104, Feb. 1997.

[9] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[10] M. S. Turan, E. Barker, W. Burr and L. Chen, "Recommendation for Password-Based Key Derivation," NIST SP 800-132, Dec. 2010.

[11] OWASP Foundation, "OWASP Top 10." [Online]. Available: https://owasp.org/www-project-top-ten/

[12] W. Stallings, Cryptography and Network Security: Principles and Practice, 7th ed. Pearson, 2017.

[13] Python Cryptographic Authority, "cryptography" library documentation. [Online]. Available: https://cryptography.io

[14] United Nations, "The 17 Sustainable Development Goals." [Online]. Available: https://sdgs.un.org/goals

[15] Pallets, "Flask documentation." [Online]. Available: https://flask.palletsprojects.com

Appendix A: Output of demo.py

```
== 1. Normal journey
registered SPT-5A208B33, label code E88CF00A7C
  #1 REGISTERED by acme_factory  @ Noida Plant  hash=9c3c7f7ba490...
  #2 SHIPPED    by acme_factory  @ Noida Plant  hash=481924c9f30f...
  #3 RECEIVED   by delhi_hub     @ Delhi Hub    hash=612afc6edde6...
  #4 SHIPPED    by delhi_hub     @ Delhi Hub    hash=99a821a3729e...
  #5 RECEIVED   by city_store    @ Store 12     hash=550af98de498...
  #6 SOLD       by city_store    @ Store 12     hash=cdfa177da1cd...
customer verifies: {'authentic': True, 'state': 'SOLD', 'events': 6}

== 2. Counterfeit label
{'authentic': False, 'reason': 'label code does not match (possible
counterfeit)'}

== 3. Access control
  city_store: blocked -> role 'retailer' is not allowed to 'register'
  alice: blocked -> role 'customer' is not allowed to 'ship'
  skipping steps blocked -> invalid step: REGISTERED -> SOLD

== 4. Tamper attack on the database
anchor (saved outside DB): 2ca999751e848fa82abd ...
audit ok? False
  - seq 3: content does not match its hash (edited)
customer verify now: tracking history failed integrity check
```

Appendix B: How to run the project

```
pip install -r requirements.txt
python app.py                                  # website at
http://127.0.0.1:5000
python demo.py                                 # walkthrough in the terminal
python -m unittest discover -s tests -v        # tests
python experiments/run_experiments.py          # experiments (needs
requirements-dev.txt)
```

Demo accounts (password demo1234): acme_factory (manufacturer), delhi_hub (distributor), city_store (retailer), auditor (admin), customer1 (customer).

Appendix C: Repository structure

```
secure-product-tracking/
  README.md  DEPLOY.md  LICENSE  requirements.txt  Procfile  render.yaml
  app.py  wsgi.py  demo.py  templates/  static/
  sptrack/        crypto_basics.py signing.py auth.py ledger.py service.py
demo_data.py cli.py
  tests/          test_crypto_basics.py test_tracking.py test_web.py
  experiments/    run_experiments.py make_diagrams.py take_screenshots.py
results/
  docs/           DESIGN.md REFERENCES.md demo_output.txt screenshots/
```